

Task 2 Report

Task Title: Cross-Site Scripting (XSS) Attack and Defense

Target URL: <http://testphp.vulnweb.com/>

1. Objective

To demonstrate a **Cross-Site Scripting (XSS)** vulnerability by injecting malicious scripts into input fields and URL parameters on a web application, then propose and implement defenses against it.

2. Methodology

◆ 2.1 Target Page 1:

URL Tested:

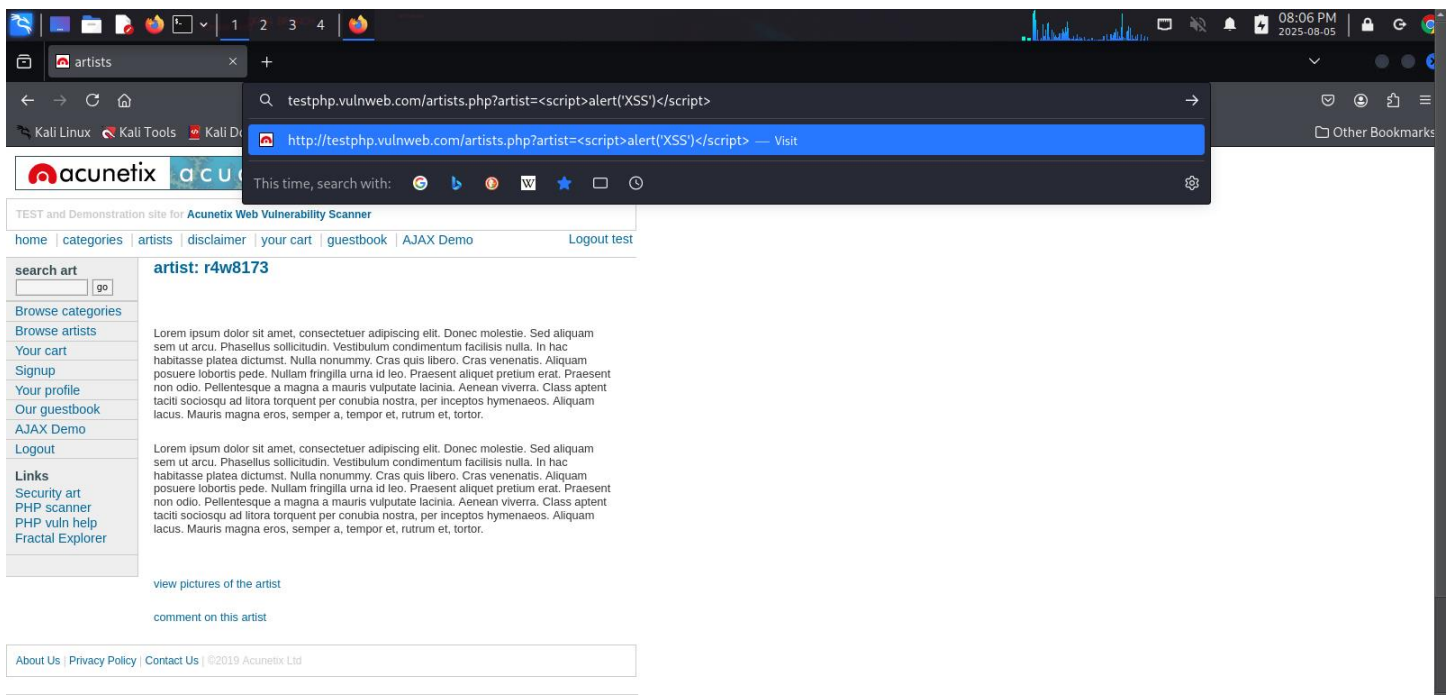
<http://testphp.vulnweb.com/artists.php?artist=>

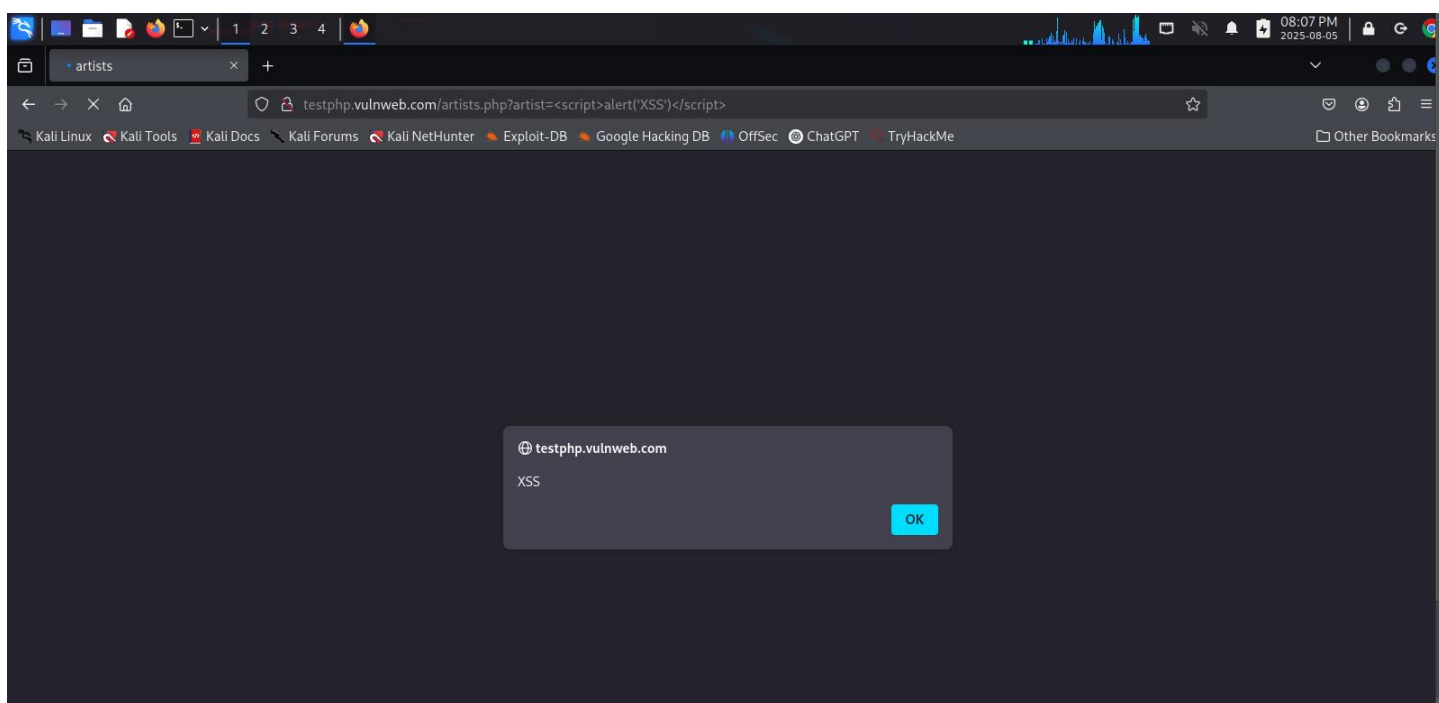
◆ 2.2 Payload Used:

```
<script>alert('XSS')</script>
```

◆ 2.3 Result:

The payload successfully triggered a browser alert, confirming **reflected XSS vulnerability** via URL parameters.





◆ 2.4 Target Page 2:

URL Tested:

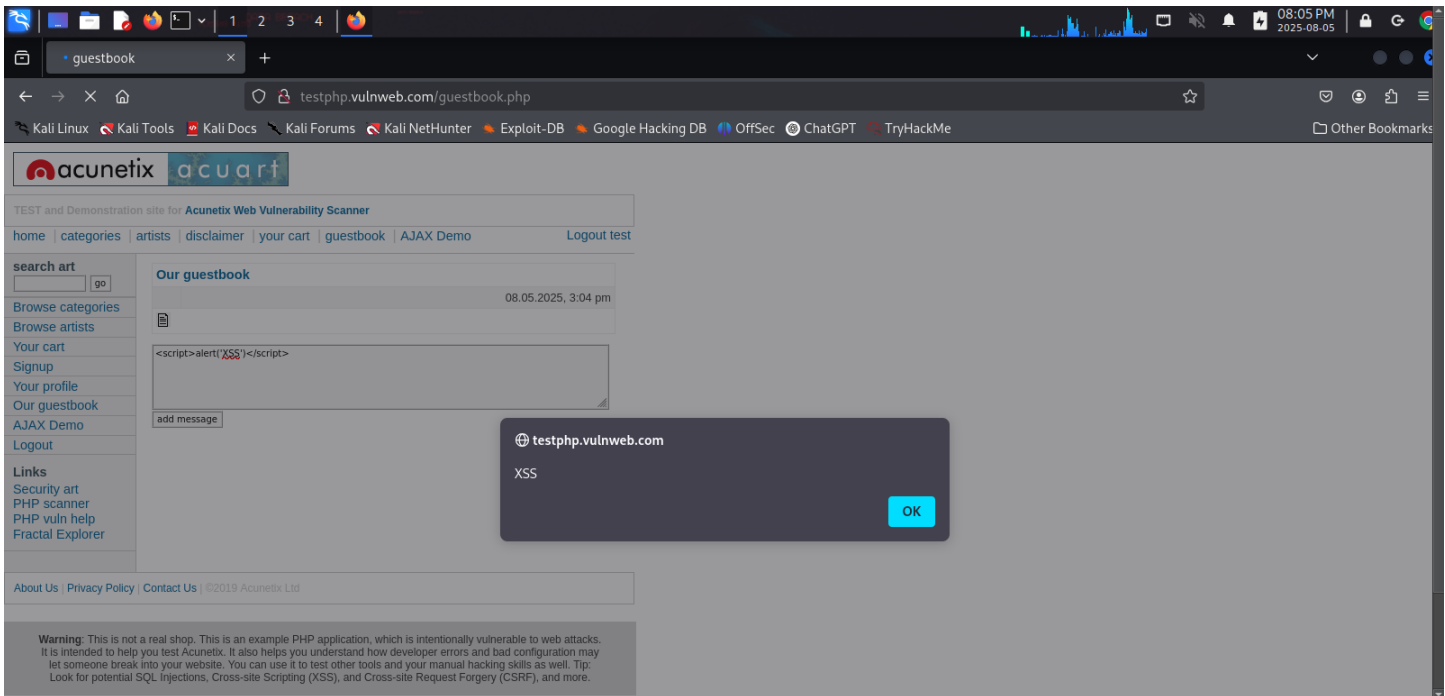
<http://testphp.vulnweb.com/guestbook.php>

◆ 2.5 Payload Used in Input Field:

```
<script>alert('XSS')</script>
```

◆ 2.6 Result:

The script executed after submission, indicating the page **stores and reflects** malicious input — a combination of **stored** and **reflected XSS**.



⚠️ 3. Impact

If exploited by an attacker, XSS can lead to:

- Cookie/session theft
- Credential hijacking
- Fake login forms (phishing)
- Browser-based malware injection
- Defacement or control of user interface

✅ 4. Recommended Fixes & Defensive Implementation

To mitigate XSS:

🔒 4.1 Input Sanitization

- Escape special characters like <, >, ', ", /, etc.
- Remove script tags using server-side filtering.
- Convert < to <, > to >, etc.

🔒 4.2 Server-side Validation (PHP Example)

```
function sanitizeInput($data) {  
    return htmlspecialchars(strip_tags($data), ENT_QUOTES, 'UTF-8');  
}
```

// Usage

```
$user_input = $_GET['artist'];  
$safe_input = sanitizeInput($user_input);  
echo "Artist: " . $safe_input;
```

4.3 Use Content Security Policy (CSP)

To restrict script execution:

```
Content-Security-Policy: default-src 'self'; script-src 'self';
```

4.4 Use Web Application Firewalls (WAFs)

A WAF can help detect and block XSS patterns.

5. Conclusion

Both reflected and stored XSS vulnerabilities were identified on the target web application. The site does not properly sanitize or validate user inputs. The issue can be mitigated through robust server-side sanitization, encoding output, and applying strict content policies.
